

Cloud Computing & Container – Cheat Sheet

1. Cloud Computing Grundlagen

Was ist Cloud Computing? Modell für einfachen, bedarfsorientierten Netzwerkzugriff auf gemeinsam genutzte IT-Ressourcen (Server, Speicher, Apps), die schnell bereitgestellt und freigegeben werden können – mit minimalem Verwaltungsaufwand.

Cloud-Schnittstellen: CLI · Web-UI · API

5 Merkmale einer Cloud

1. **On-Demand Self-Service** – Ressourcen selbst bereitstellen, ohne den Anbieter zu kontaktieren.
2. **Broad Network Access** – Zugriff über das Internet von beliebigen Geräten.
3. **Resource Pooling** – Hardware wird von vielen Kunden gemeinsam genutzt (Multi-Tenancy). Standort meist unbekannt, nur Region wählbar. → *Economy of Scale, Overcommitment* (Anbieter verkauft mehr als physisch vorhanden).
4. **Rapid Elasticity** – Ressourcen schnell hoch- und runterskalieren (z. B. Black Friday).
5. **Measured Service** – Verbrauch wird gemessen → Pay-as-you-go.

Service-Modelle (klären die Verantwortlichkeiten / Shared Responsibility)

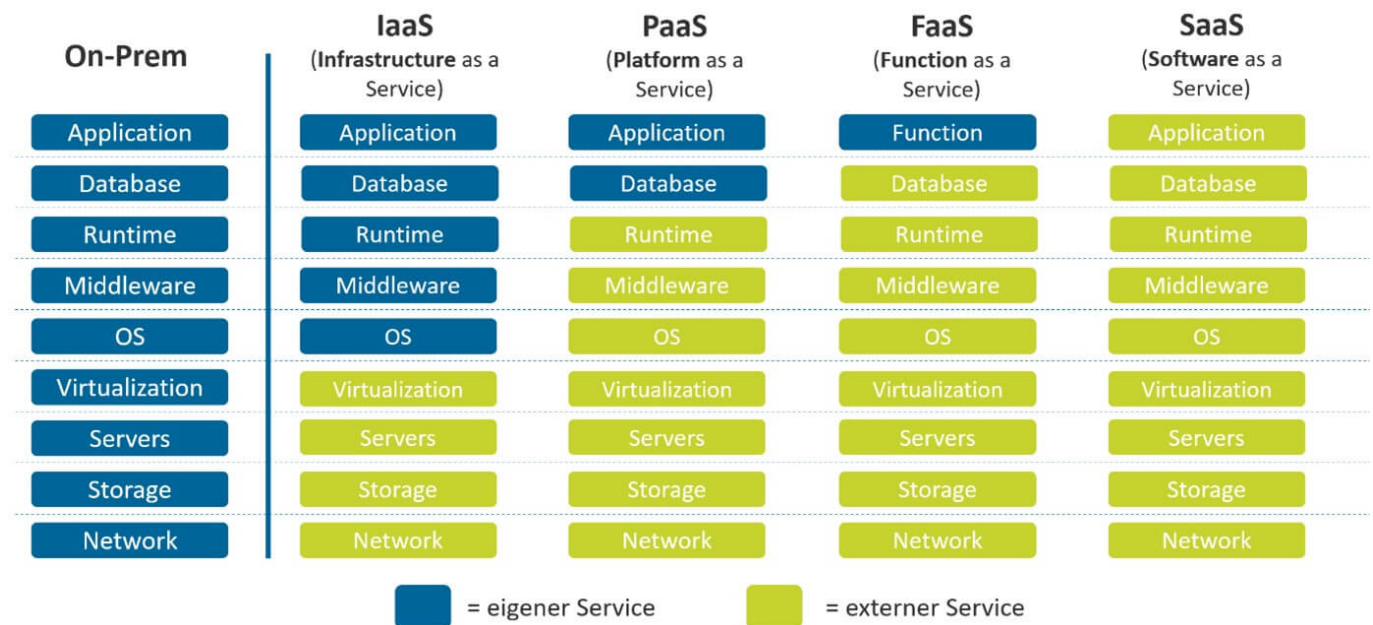


Figure 1: modelle

Modell	Du verwaltest	Anbieter verwaltet	Beispiele
IaaS	OS, Middleware, Apps, Daten	Netzwerk, Storage, Server, Virtualisierung	AWS EC2, Azure VMs
PaaS	Apps, Daten	+ Runtime, Middleware, OS	Heroku, Google App Engine
SaaS	(nur Login/Einstellungen)	Alles	Gmail, Netflix, M365

Modell	Du verwaltest	Anbieter verwaltet	Beispiele
FaaS	Nur die Funktion	Alles andere	AWS Lambda, Azure Functions

Shared Responsibility: Anbieter = Security **OF** the Cloud (Hardware, Gebäude, Hypervisor). Kunde = Security **IN** the Cloud (Daten, User, Berechtigungen, Apps). Je höher die Abstraktion (→ SaaS), desto weniger Verantwortung trägt der Kunde.

Deployment-Modelle

Modell	Beschreibung
Private Cloud	Nur ein Kunde, volle Kontrolle, eigene Hardware möglich. Teuerste Option.
Public Cloud	Für alle zugänglich. Kein Zugang zu Servern/Gebäuden. Günstig, flexibel.
Community Cloud	Mehrere Organisationen mit gleichem Interesse teilen sich die Cloud.
Hybrid Cloud	Kombination mehrerer Modelle (meist Private + Public). Beste aus beiden Welten.

Vorteile / Nachteile der Cloud

Vorteile	Nachteile
Keine Hardware-Kosten (Pay-as-you-go)	Internet-Abhängigkeit
Schnelle Bereitstellung	Vendor Lock-in
Hohe Skalierbarkeit & Verfügbarkeit	Datenschutz (DSGVO vs. US-Server)
Weniger Wartungsaufwand	Kostenkontrolle (unbemerkt hohe Rechnungen)

Monitoring vs. Logging

Monitoring = Echtzeitdaten (Metriken: CPU, RAM, Netzwerk). Beantwortet: *Dass* ein Problem existiert.

Logging = Einzelne Events aufzeichnen (Fehler, Logins, Abstürze). Beantwortet: *Warum* ein Problem existiert.

2. Cloud-Anbieter

Hyperscaler: AWS · Microsoft Azure · Google Cloud (GCP)

Europäische Alternativen: Hetzner · OVHcloud · Swisscom Cloud

Kriterium	AWS	Azure	GCP
Fokus	Marktführer, grösstes Angebot	Microsoft-Integration, Enterprise	Kubernetes, Big Data, AI
Compute	EC2	Virtual Machines	Compute Engine
Kubernetes	EKS	AKS	GKE
Storage (Objekt)	S3	Blob Storage	Cloud Storage
DB (SQL / NoSQL)	RDS · Aurora / DynamoDB	Azure SQL / Cosmos DB	Cloud SQL / Bigtable
AI / ML	Bedrock / SageMaker	Azure OpenAI	Vertex AI
Stärke	Grösste Auswahl, reife Plattform	Perfekt für MS-Umgebungen	Kubernetes & Big Data

3. Container-Technologie

Was ist Container-Technologie? Anwendungen werden inklusive Abhängigkeiten in einer isolierten Einheit verpackt. Container teilen sich den **Kernel des Host-Betriebssystems** (isoliert via Linux-Namespaces & cgroups). Kein vollständiger Computer wird simuliert.

VM vs. Container

Kriterium	Virtuelle Maschine (VM)	Container
Betriebssystem	Eigener Kernel (Guest OS)	Teilt Host-Kernel
Startzeit	Minuten	Sekunden / Millisekunden
Speicher	GB	MB
Effizienz	Geringer (Hardware-Emulation)	Sehr hoch (direkt auf Kernel)
Portabilität	Begrenzt	Sehr hoch ("Run anywhere")
Isolation	Stark (eigener Kernel)	Schwächer (geteilter Kernel)

Können VMs immer durch Container ersetzt werden? Nein! VMs sind nötig bei:

- Unterschiedlichen Betriebssystemen (z. B. Windows-App auf Linux)
- Maximaler Sicherheitsisolation (Multi-Tenancy, Banken)
- Direktem Hardware-Zugriff / Kernel-Modifikationen

Praxis: In der Cloud werden Container oft innerhalb von VMs gestartet (beste aus beiden Welten).

Bekannte Produkte

Bereich	Produkte
VM-Software	VMware vSphere/ESXi · Hyper-V · KVM · VirtualBox · Proxmox
Container Engines	Docker · Podman · containerd · CRI-O (cryo)
Orchestrierung	Kubernetes (K8s) · Docker Swarm · Docker Compose · OpenShift
Managed (Cloud)	AWS EKS · Google GKE · Azure AKS

Image · Container · Registry

Image = Read-Only Vorlage / Bauplan (enthält Code + Abhängigkeiten). Änderungen immer im **Dockerfile**, nie im laufenden Container.

Container / Pod = Laufende Instanz eines Images (mit beschreibbarer Schicht). In Kubernetes läuft ein Container in einem **Pod** (kleinste deploybare Einheit in K8s).

Registry = Zentraler Speicherort für Images (push/pull). Beispiele: Docker Hub, GitHub Container Registry.

Dockerfile = Bauplan für den Build-Prozess eines Images.

Immutable Containers (Unveränderlichkeit)

Laufende Container werden **nie** im Betrieb verändert oder gepatcht.

Workflow bei Änderungen: 1. Neues Image bauen → 2. Alten Container löschen → 3. Neuen Container starten.

Vorteile: Reproduzierbarkeit · kein Configuration Drift · einfaches Rollback.

Wichtige Daten werden über externe **Volumes** außerhalb des Containers gespeichert. Dateinamen/Verzeichnisse sind persistent.

Self-Managed vs. Fully Managed

	Self-Managed	Fully Managed (z. B. GKE, EKS, AKS)
Kontrolle	Voll	Eingeschränkt
Wartungsaufwand	Hoch (Updates, Backups, Security)	Gering (Anbieter übernimmt)
Vendor Lock-in	Gering	Höher
Kosten	Günstiger bei Know-how	Teurer

4. Container-Orchestrierung

Warum braucht man Container-Orchestrierung? In Unternehmen laufen Hunderte/Tausende Container. Ohne Automatisierung: Was passiert bei Server-Absturz? Wie skaliert man bei Ansturm? Wie verteilt man Traffic? Wie deployed man ohne Downtime?

Wie funktioniert es? → Deklaratives Modell (Soll-Zustand vs. Ist-Zustand):

1. Du definierst den Soll-Zustand in einer YAML-Datei: *“Immer 3 Instanzen der App laufen.”*
2. Orchestrator überwacht kontinuierlich den Ist-Zustand.
3. Bei Abweichung → **Auto-Healing**: Neuer Container wird automatisch gestartet.

Kubernetes macht für uns: Scheduling · Load Balancing · Traffic Routing · Ressourcenzuweisung · Container-Zustand überwachen · Auto-Healing · Auto-Scaling.

Infrastructure as Code (IaC): Infrastruktur als Code beschreiben → Tool: **Terraform**.

Skalierung

Vertikal (Scale Up) = Bestehenden Server grösser machen (mehr CPU/RAM). Limit: physische Grenze.

Horizontal (Scale Out) = Weitere Instanzen / Container hinzufügen. Standard in der Cloud – nahezu unbegrenzt skalierbar, ausfallsicher.

Deployment-Strategien

Strategie	Beschreibung	Vorteil	Nachteil
Recreate	Alle alten Container stoppen, dann neue starten	Keine Versionsvermischung	Downtime
Rolling Update	Schrittweiser Austausch (Standard)	Keine Downtime	Kurzzeitig 2 Versionen gleichzeitig
Blue-Green	Zwei identische Umgebungen, Traffic umschalten	Keine Downtime, schnelles Rollback	Doppelte Infrastrukturkosten
Canary	Neue Version für kleine Nutzergruppe (z. B. 5 %)	Minimales Fehlerrisiko	Komplexes Routing

Troubleshooting in Kubernetes: Deployment · Events · Pod

5. Git & Docker Workflow

Beispiel-Dockerfiles

Standard (Docker / lokal):

```
FROM nginx

COPY apps/web /usr/share/nginx/html

# docker build -f Dockerfile -t 3.2_erster_webserver .
```

OpenShift (unprivileged – kein Root):

```
FROM nginxinc/nginx-unprivileged

COPY apps/web /usr/share/nginx/html

# docker build -f Dockerfile.oc -t ghcr.io/laurens-hertzer/html-openshift-app:v1 .
```

Unterschied: nginx-unprivileged läuft ohne Root-Rechte → Pflicht in OpenShift. FROM = Basis-Image, COPY = Dateien ins Image kopieren.

Git Workflow

```
git clone <url>      # Repository herunterladen
git pull             # Neueste Änderungen holen

git add .            # Alle Änderungen stagen

git commit -m "initial" # Commit erstellen

git push             # Änderungen hochladen
```

Docker Workflow

```
# Image bauen (tag = ghcr.io/<username>/html-docker-app:1.0.0)
docker build -f Dockerfile.web -t ghcr.io/<username>/html-docker-app:1.0.0 .

# Image hochladen
docker push <tag>

# Container starten (-d = detached, -p = Port-Mapping)
docker run -d -p 8080:80 --name web <tag>

# Container stoppen / löschen
docker stop <name>
docker rm <name>

# Image löschen
docker rmi <i-name>

# Container anzeigen (alle, auch gestoppte)
docker ps -a

# Logs anzeigen
docker logs <name>

# Shell im Container öffnen
docker exec -it <name> sh    # oder: bash
```

6. Schnell-Referenz (Prüfungs-Merksätze)

Begriff	Definition
Monitoring	Echtzeit-Metriken (DASS ein Problem existiert)
Logging	Einzelne Events aufzeichnen (WARUM es existiert)
Dockerfile	Bauplan für den Build-Prozess eines Images
Image	Read-Only Vorlage/Bauplan
Container / Pod	Laufende Instanz eines Images; in K8s läuft Container im Pod
Registry	Speicherort für Images (Docker Hub)
Scale Out	Horizontal – mehr Instanzen hinzufügen
Scale Up	Vertikal – Server grösser machen
Immutable	Container werden nicht verändert, neu gebaut
IaC / Terraform	Infrastruktur als Code verwalten
Anbieter-Verantwortung	Security OF the Cloud
Kunden-Verantwortung	Security IN the Cloud
Kubernetes	Scheduling + Load Balancing + Auto-Healing + Scaling
OpenShift	Kubernetes-Erweiterung für Unternehmen
Overcommitment	Anbieter verkauft mehr Ressourcen als physisch vorhanden
Economy of Scale	Kostenvorteile durch gemeinsame Ressourcennutzung
FaaS	Function as a Service (AWS Lambda, Azure Functions)